

Para empezar este desafío bien, tenemos que entender cómo van a ser nuestras fichas y cómo las vamos a mover en su tablero. Primero elegimos cuáles son los caracteres que queremos que tenga cada una de nuestras fichas, pero que sean muy reconocibles entre sí, para poder tener un juego más cómodo.

Fichas= (? ! + # \$ %) (? = 000, ! = 001, + = 010, # = 011, \$ = 100, % = 101,) Estas son las que usaremos en este desafío con su respectiva asignación de bits.

Después vamos a entender bien qué es lo que tenemos que hacer con estas. Primero sabemos que no se van a mover las fichas a mano, las únicas acciones que tiene el jugador son: añadir fila, añadir columna y eliminar ficha, con esto ya podemos empezar a mirar un poco más adentro del código para estructurar nuestro juego.

Hay que saber que tenemos fichas que ocupan 3 bits cada 1, y que estos se manejan igual que todos los bits, por lo tanto podemos decir que tendríamos $2^3 = 8$ combinaciones, 6 serían para las fichas en cada casilla y las otras 2 representan estado vacío (110) y un estado especial (111) como nos lo muestran en el documento.

Aparte de esto, tenemos que conocer la representación del tablero, que vamos a hacer con una secuencia completa de bits. El tablero tiene F filas y C columnas y nos dicen que la cantidad de bits requeridos son $3 \times F \times C$.

Todo esto nos lo dice básicamente el documento, pero ahora tenemos que plantear cómo lo vamos a resolver en realidad. Hay que tener en cuenta que necesitamos la cantidad mínima de bytes para almacenar los bits, entonces vamos a empezar por resolver esto:

Sabemos que los bits totales son $3 \times F \times C$ y que la memoria en C++ es reservada en bytes, como cada byte es equivalente a 8 bits:

bytes mínimos = $(3 \times F \times C) / 8$

Esto lo tenemos que pasar a C++, pero tenemos que usar divisiones enteras y enteros, por lo tanto tenemos que pensar en una forma de guardarlos incluso si se “saltan” los bits a los otros bytes:

Como sabemos que en C++ la división entera toma el valor sin los decimales, sin importar que tan cerca estén del número siguiente, entonces habría que buscar una forma de redondear hacia arriba, para que los bits si se guarden bien, pues 3 no es divisor de 8 y sabemos que 3 bits/1 byte no es exacta.

Para hallar la cantidad de bytes totales a reservar, si se le suma la cantidad de bytes que necesitamos -1 (tamaño del byte-1) haciendo que se redondee hacía abajo con la división entera.

Esto está pensado así porque son operaciones básicas, pues la otra opción es usar funciones, que es menos eficiente para este caso.

Así se verá en C++:

```
int total_fichas = F * C;  
int bits_totales = total_fichas * 3;  
int bytes_necesarios = (bits_totales + 7) / 8;
```

```
unsigned char* tablero = new unsigned char[bytes_necesarios]
```

Ahora tenemos que pensar en cómo organizar las fichas, pues ya tenemos el tamaño del tablero. Como los bytes van del 0 al 7 y tenemos fichas de 3 bits, las fichitas no van a coincidir con el límite del byte. N va desde 0 hasta infinito.

Para el bit inicial: **bit_inicial** = $N * 3$, donde **N** = (fila * C) + columna

El arreglo de memoria unsigned char* está en secuencia como nos dijeron en el documento, es una secuencia de bytes y se necesita saber en cuál de todos los bytes va a estar la ficha que buscamos. Para esto necesitamos el bit inicial en división entera con 8, entonces el byte buscado será: **byte_buscado = bit_inicial/8**

Aparte del byte buscado, tenemos que saber en qué parte dentro de este está la ficha y sabemos que dentro del byte hay 8 posiciones, entonces el bit de inicio estará en una de estas. **desplazamiento = bit_inicial%8**, esto nos determina donde inicia la lectura o escritura de los 3 bits de la ficha.

Ya tenemos como se “buscan” las fichas dentro de los bytes, pero se tienen tres casos, los cuales son:

Caso 1: Si está en las posiciones desde el 0 al 5 estarán en 1 solo byte.

Este caso se soluciona corriendo los bits al inicio para alinearlos y “limpiar” los otros bits que no nos interesan, consiguiendo la ficha.

Caso 2: Si hay 2 bits al final y uno en el byte siguiente.

Hay que leer los últimos 2 bits del bloque de byte y el primero de la siguiente, para luego “pegarlos” y así tendríamos la ficha.

Caso 3: Si hay 1 bit al final y 2 en el byte siguiente.

Eso sería lo mismo básicamente, se lee el último bit y los 2 primeros de la siguiente, después se pegan y así conseguimos la ficha.

Estos 3 se tienen que hacer con operaciones de bits, las cuales vamos a decidir en medio de la programación

Caída y generación de fichas (reorganización del tablero):

Caída de fichas: se recorre el tablero columna a columna de abajo a arriba. Si encuentra una casilla vacía busca la ficha superior más cercana y la hace caer.

Generación de fichas: Las posiciones vacías que quedan en la parte superior se rellenan con números aleatorios de las fichas.

Gestión de memoria dinámica:

Cómo sabemos, si añadimos filas o columnas no se mantiene un tamaño estático. Para poder manejar esto, vamos a usar la manera de crear un nuevo arreglo para reubicar las fichas y borrar el viejo, ya que en este desafío si se puede usar de esta manera. Tendríamos que acomodar las fichas hasta el inicio del arreglo para cerrar los huecos que nos queden y ya cuando tengamos la memoria menor al 65% se crea un arreglo más pequeño para meter lo que ya tenemos en ese y luego liberar.

Combinaciones y Cascadas:

Se analiza la matriz buscando secuencias de 3 o más fichas iguales en dirección horizontal y vertical, luego las fichas pertenecientes a un trío se marcan como vacías. Si se realizan eliminaciones, se llama a la función de caída y relleno. Por último se vuelve a ejecutar la detección de tríos automáticamente hasta que no se formen más combinaciones, registrando el conteo de cascadas y sumando el puntaje del jugador.

Todo esto hay que hacerlo ya en código, manteniendo las pautas que nos dieron los profesores y teniendo en cuenta la eficiencia en el programa, sabiendo que estamos trabajando en nivel de bits y que es un nivel más bajo de programación.